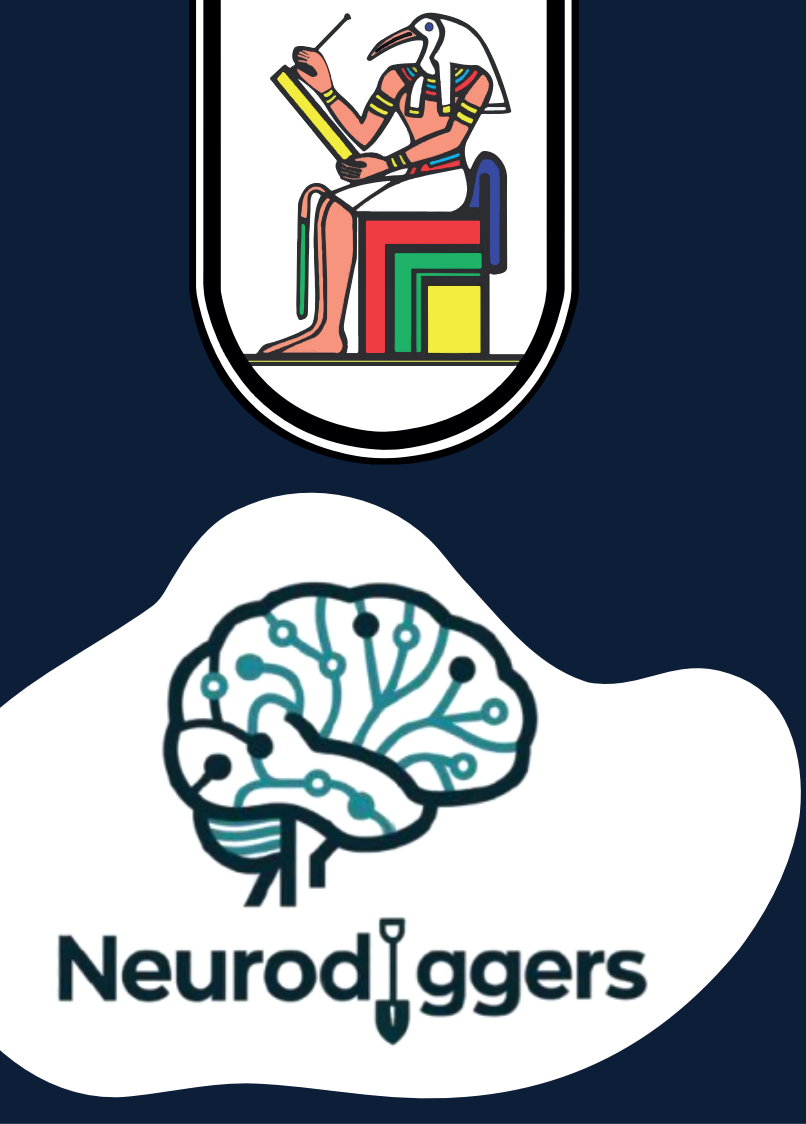




Physics-Informed Neural Networks for Predictive Vehicle Control

MTHG103E

Supervised by Dr. Samah Tantawy



Digging deep into intelligence & driving the future

Abstract

Model predictive control (MPC) needs a prediction model that is both fast and accurate. We train a physics-informed neural network for control (PINC) on a vehicle model with coupled speed and steering dynamics and use it inside MPC. Against the same network trained on data alone, the physics term made the learned dynamics **13×** more accurate and cut 50-step prediction errors **2.2×** (**17×** with only 100 training trajectories). In closed loop, PINC-MPC tracked as well as an MPC using the exact model over four manoeuvres and **30** noisy runs each.

Problem Definition

Road crashes kill **1.19 million** people a year [1], most through human error, which makes automated driving a safety priority. Automated vehicles are often controlled with **model predictive control (MPC)**: every 0.1 s it predicts the vehicle's motion and optimises force and steering. MPC is only as good as its prediction model, which must be **accurate, fast** and, for real vehicles, **learnable from data**. Exact models can be slow or unknown; purely data-driven networks need large datasets and drift over long horizons. Can a physics-informed network give MPC all three?

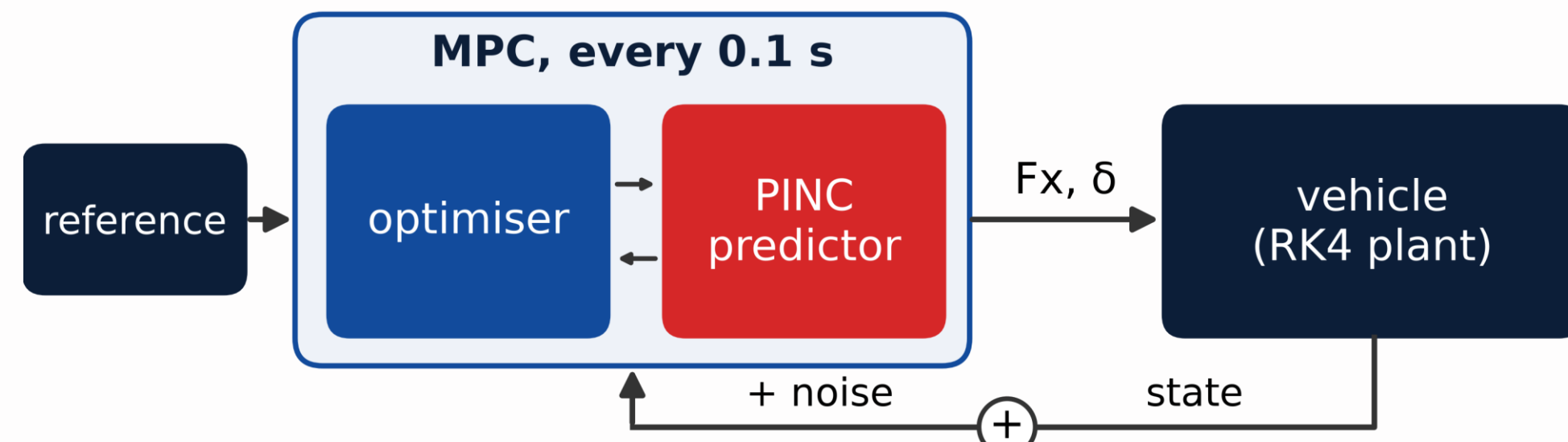


Figure 1. The simulated loop; the predictor is PINC, the exact model, the data-only network or a linearisation.

Literature Review

Physics-informed neural networks (PINNs) [2] fit data while penalising violations of the governing differential equations. **PINC** [3] extends them to control: time, initial state and input are network inputs, so one trained network predicts the state over a control period and can be chained inside MPC. PINN-based MPC has been used for trajectory tracking of guided vehicles [4] and of vehicles with noisy measurements [5]; neither compares against the exact model with the same solver, or against the same network trained without physics. Sliding mode [6], H_∞ [7] and min-max MPC [8] treat model uncertainty in the design; we do not use them and instead test robustness by perturbing the plant (Table 3).

What this work adds

1. A PINC surrogate of a single-track model with coupled longitudinal and lateral dynamics, with an exact initial condition and a per-state output scaling D that balances the physics terms (Figure 3).
2. Evidence over five seeds that the network learns the dynamics, not only end states, from 10^2 to 2×10^4 training trajectories (Table 1, Figure 5).
3. A controlled benchmark: exact, PINC, data-only and linearised models inside the same MPC and plant, 4 manoeuvres $\times$ 30 runs and 8 plant mismatches (Tables 2–3).

Mathematical Modeling

We describe the vehicle with a single-track (bicycle) model [9]: longitudinal speed v_x , lateral speed v_y , yaw rate r and heading ψ , driven by a longitudinal force F_x and steering angle δ . The tyre forces depend on the front and rear slip angles, so speed and steering are coupled. **PINC** learns this model and replaces it inside the **MPC**.

Longitudinal dynamics:

$$m(\dot{v}_x - v_y r) = F_x - \frac{1}{2} \rho C_d A v_x^2 - F_{rr} - F_{yr} \sin \delta$$

Lateral and yaw dynamics:

$$m(\dot{v}_y + v_x r) = F_{yr} \cos \delta + F_{yr}$$

$$I_z \dot{r} = l_f F_{yr} \cos \delta - l_r F_{yr}$$

MPC cost function:

$$J = \sum_{k=1}^N e_k^T Q e_k + \sum_{k=0}^{N-1} \tilde{u}_k^T R \tilde{u}_k + \Delta \tilde{u}_k^T R_\Delta \Delta \tilde{u}_k$$

$$s. t. \quad S_{k+1} = f_w(T, S_k, u_k), \quad u_{\min} \leq u_k \leq u_{\max}$$

PINC surrogate (Antonelo et al. [3]):

1. A neural network trained with a physics-informed loss maps time t , initial state and input u to the predicted state; chaining it over the horizon gives the MPC its model.
2. Exact initial condition and a per-state output scaling D, which keeps the four physics terms balanced:

$$\hat{S}(t) = \hat{S}_0 + \int_0^t D f_w(s, t, S_0, u), \quad D = \text{diag}(S_f T / S_x)$$

Experimental Setup

Approach

- One **MPC**, four prediction models: exact model, **PINC**, data-only network, linearised model.
- Data-only network: the same network trained without the physics term ($\lambda = 0$).

Neural network architecture

- **7** inputs: time, initial state (v_x, v_y, r, ψ), force, steering
- **8** hidden layers $\times$ **64** neurons, tanh
- exact initial condition, per-state output scaling D
- **4** outputs: v_x, v_y , yaw rate r , heading ψ

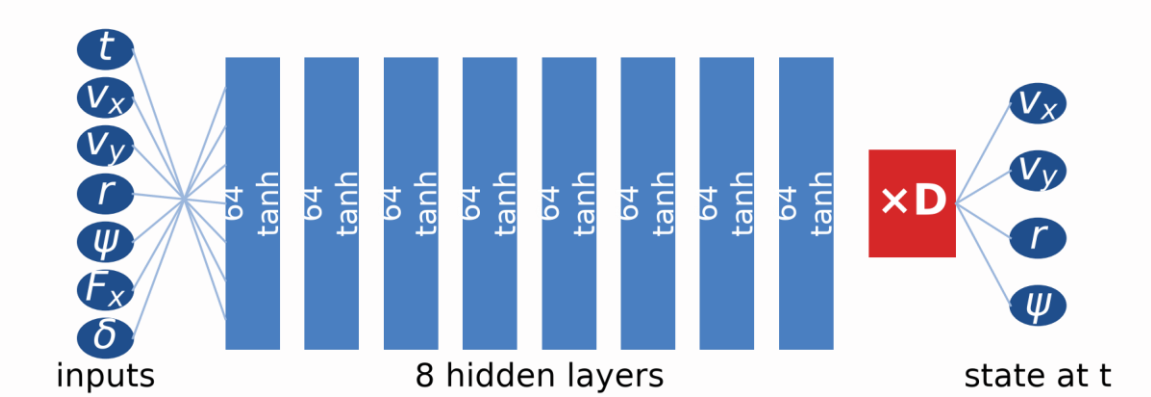


Figure 2. PINC network.

Training protocol

- loss $L = L_{\text{data}} + \lambda L_{\text{phys}}$, $\lambda = 0.01$
- **Adam** ($\text{lr} = 0.001$), then **L-BFGS**
- **20 000** simulated trajectories
- per-state normalisation:

	scale S_x, S_u	rate S_r
v_x	30 m/s	2 m/s ²
v_y	1.5 m/s	9.4 m/s ²
r	0.6 rad/s	5.3 rad/s ²
ψ	0.5 rad	0.35 rad/s
F_x, δ	6000 N, 0.3 rad	–

Results

Does PINC learn the physics? (5 training seeds)

normalised error	data-only	PINC	PINC gain
20 000 training trajectories			
dynamics dx/dt	$1.4 \cdot 10^{-1}$	$1.0 \cdot 10^{-2}$	13×
one step (0.1 s)	$9.3 \cdot 10^{-4}$	$7.9 \cdot 10^{-4}$	1.2×
10 steps (1 s)	$6.0 \cdot 10^{-3}$	$3.4 \cdot 10^{-3}$	1.7×
50 steps (5 s)	$2.5 \cdot 10^{-2}$	$1.1 \cdot 10^{-2}$	2.2×
50 steps, unseen speeds	$9.5 \cdot 10^{-2}$	$3.7 \cdot 10^{-2}$	2.6×
100 training trajectories			
one step (0.1 s)	$2.9 \cdot 10^{-2}$	$2.0 \cdot 10^{-3}$	14×
50 steps (5 s)	$2.3 \cdot 10^{-1}$	$1.3 \cdot 10^{-2}$	17×

Table 1. PINC beats the identical network trained without physics in every metric and every seed; the gain is largest with little data.

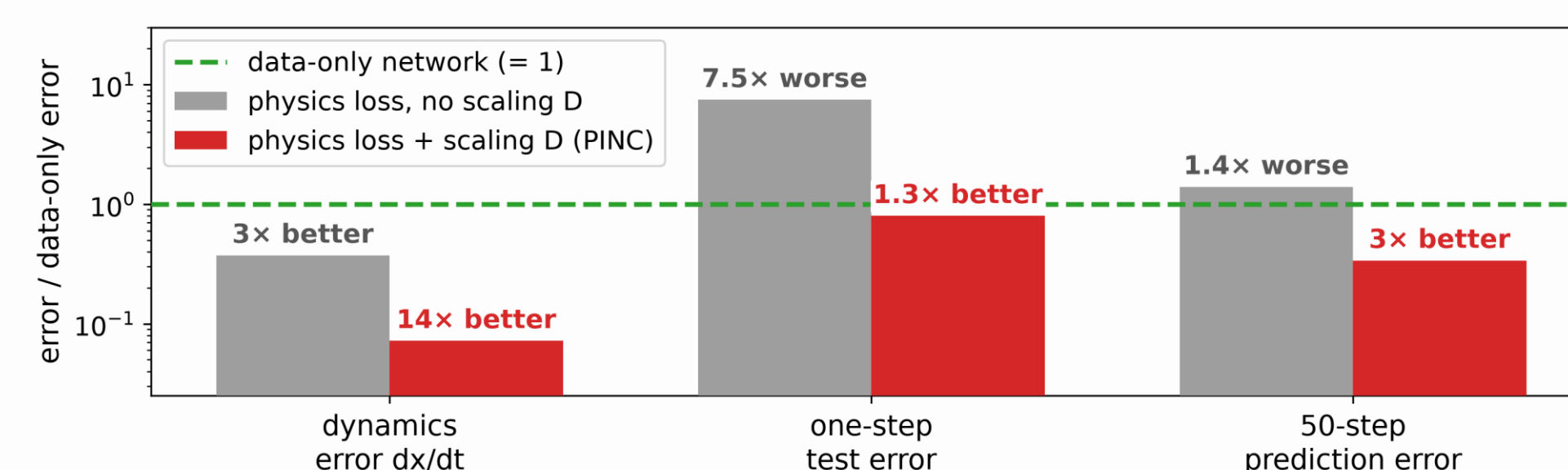


Figure 3. Without the per-state scaling D the physics loss makes the network 7.5× less accurate; with D it improves every error (seed 0).

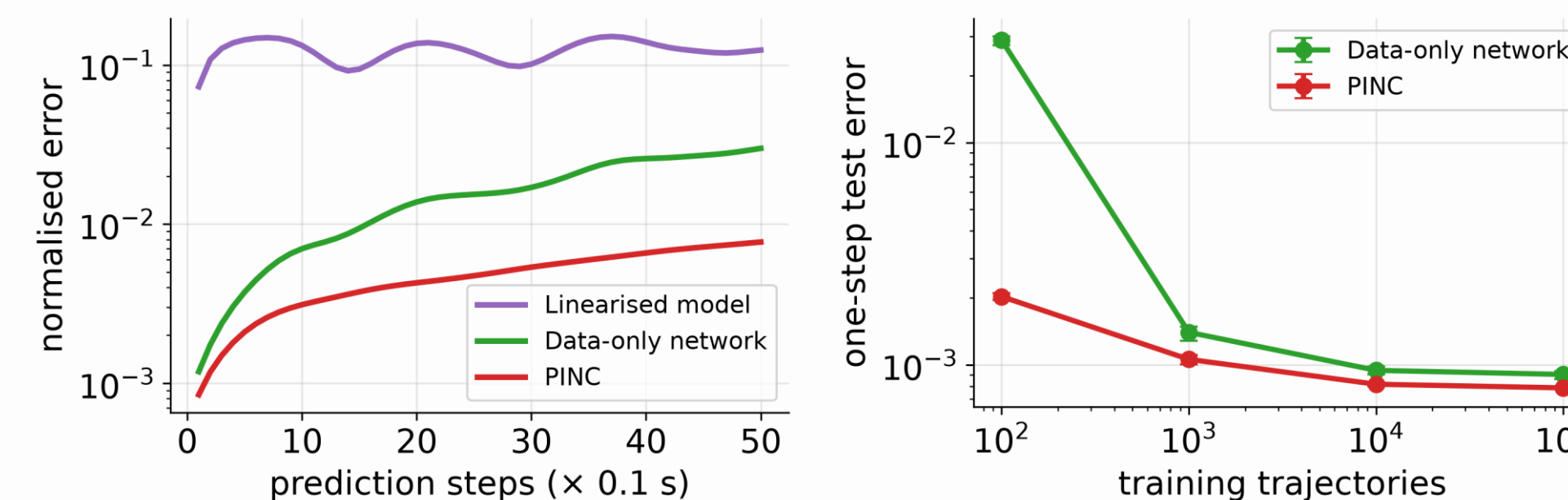


Figure 4. Error vs horizon (one model).

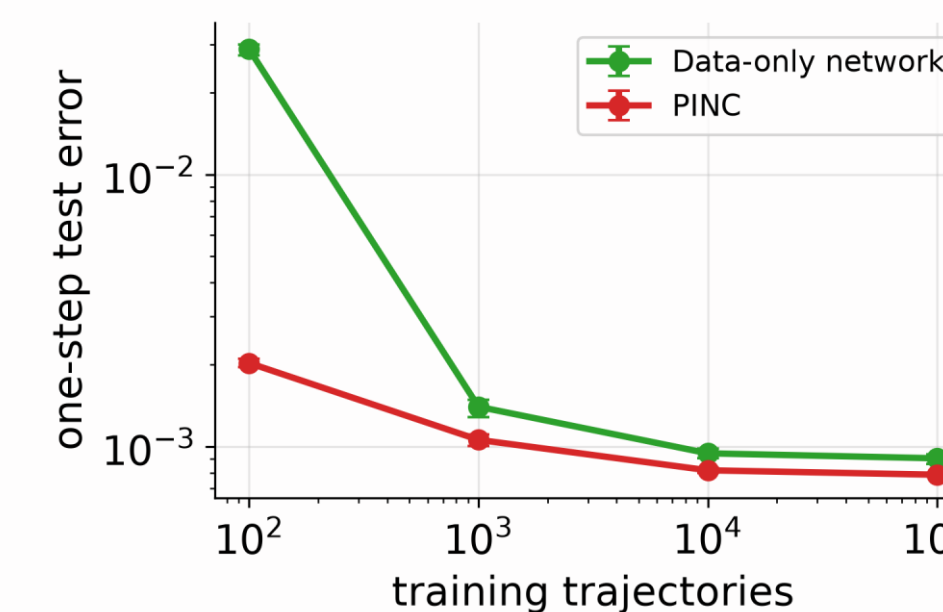


Figure 5. Error vs training data (5 seeds).

Closed-loop control (4 manoeuvres $\times$ 30 noisy runs)

RMSE, mean of 30 runs	exact	PINC	data-only	linear
nominal plant	5.8	5.8	8.8	5.8
speed sine: speed [cm/s]	44.9	44.7	44.0	69.6
lane change: lateral [cm]	3.3	3.3	3.4	3.3
lane change: speed [cm/s]	4.6	4.6	7.7	4.6
double lane ch.: lateral [cm]	14.3	14.4	14.3	14.3
solve time, N = 10 [ms]	10.6	10.0	9.6	20.1
solve time, N = 40 [ms]	47.5	32.4	29.6	41.3

Table 2. PINC-MPC tracks like the exact-model MPC in every manoeuvre (all solves converged); the data-only MPC drifts in speed. Solve time: median, one CPU thread.

plant mismatch (lane change)	exact	PINC	data-only
nominal plant	3.3 / 4.6	3.3 / 4.6	3.4 / 7.7
mass -20 %	3.6 / 4.4	3.7 / 4.4	3.7 / 7.6
mass +20 %	4.2 / 4.9	4.1 / 4.9	4.3 / 7.9
tyre stiffness -30 %	6.2 / 4.6	6.1 / 4.7	6.2 / 7.9
tyre stiffness +30 %	3.8 / 4.6	3.9 / 4.6	3.9 / 7.6
friction $\mu = 0.7$	4.3 / 4.6	4.2 / 4.6	4.3 / 7.8
friction $\mu = 0.4$	7.5 / 4.8	7.3 / 4.8	7.5 / 8.1
-1 kN force disturbance	3.3 / 12.0	3.3 / 12.2	3.4 / 8.4

Table 3. Lateral [cm] / speed [cm/s] RMSE when the plant differs from the controllers' model: PINC degrades like the exact model (the data-only bias happens to offset the force step).

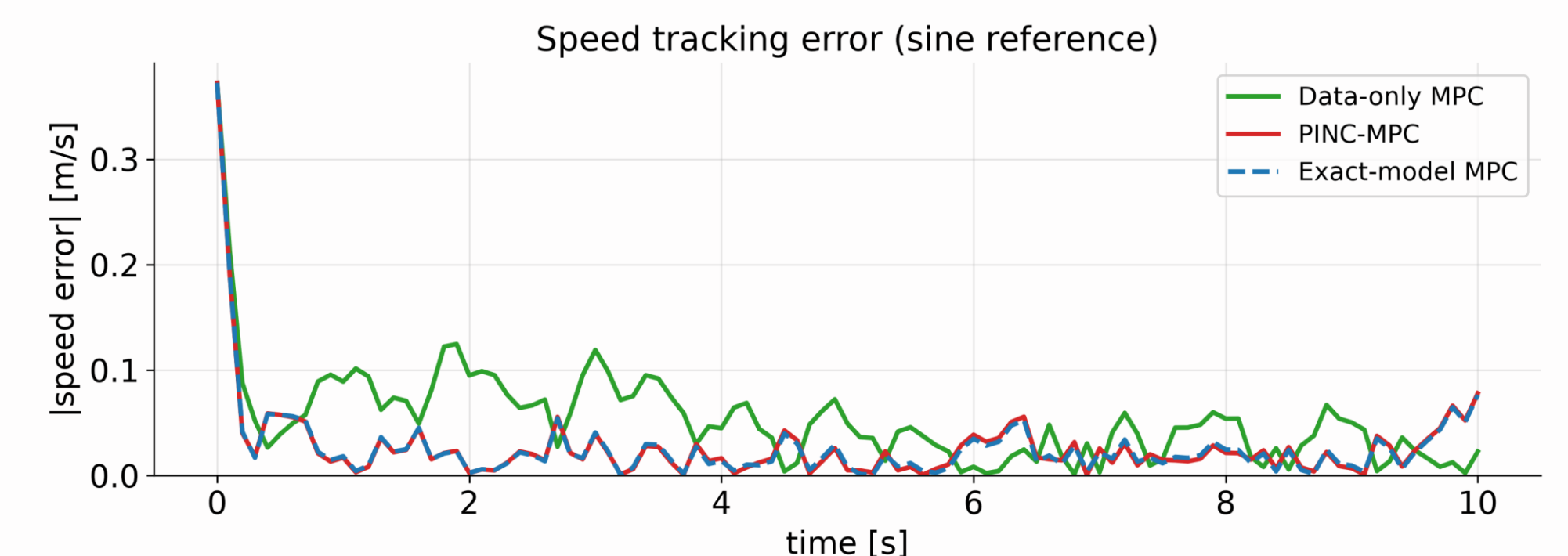


Figure 6. Speed error on the sine reference (run 13, the median run).

Conclusion

Scaling the physics loss per state made the network learn the vehicle dynamics (13× lower dynamics error than without physics) instead of only fitting data; inside MPC it controls as well as the exact model. Without the scaling, physics made the model less accurate. Next steps:

1. Unknown vehicles: learn from measured data, with physics as a prior.
2. Higher-fidelity vehicle models, where the surrogate's speed advantage grows.
3. Hardware-in-the-loop and on-vehicle tests.

References

- [1] World Health Organization, Road traffic injuries, fact sheet, 2023.
- [2] M. Raissi, P. Perdikaris, G. E. Karniadakis, J. Comput. Phys. 378:686–707, 2019.
- [3] E. A. Antonelo et al., Neurocomputing 579:127419, 2024.
- [4] Y. Li, L. Liu, World Electr. Veh. J. 15(10):460, 2024.
- [5] L. Jin et al., IEEE Trans. Intell. Veh. 9(1):1305–1319, 2024.
- [6] K. D. Young, V. I. Utkin, U. Özgüner, IEEE Trans. Control Syst. Technol. 7(3):328–342, 1999.
- [7] F. Oudjama et al., J. Intell. Syst. Control 2(3):170–182, 2023.
- [8] J. Zhou et al., IEEE Trans. Intell. Transp. Syst., 2022.
- [9] R. Rajamani, Vehicle Dynamics and Control, 2nd ed., Springer, 2012.



References

Authors:

Ahmed H. Labib
Ahmed W. ElMarsafy
Ziad M. ElGendy

Zeyad E. Farghaly
Ziad H. Karawia
Zeyad M. ElSonney

Mohammed D. ElHosary

